HZ BOOKS
华章科技

PEARSON

华章专业开发者丛书

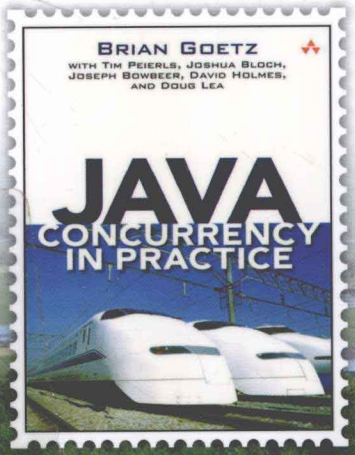
本书曾获Jolt大奖提名
JavaOne大会最畅销图书
了解Java并发编程必读佳作

Java 并发编程实战

Java Concurrency in Practice

（美） Brian Goetz
Tim Peierls
Joshua Bloch
Joseph Bowbeer
David Holmes
Doug Lea
著

童云兰 等译



机械工业出版社
China Machine Press

Java 并发编程实战 | Java Concurrency in Practice

线程是Java平台的基础组成部分之一。随着多核处理器逐渐成为主流，如何高效地使用并发已成为构建高性能应用程序的重要因素。Java SE 5和Java 6在并发程序开发方面取得了巨大的进步，在其Java虚拟机中能支持一些高性能的并且具有高可伸缩性的并发类，此外还支持一组新的并发基础构建模块。在本书中，这些新功能的编写者们不仅介绍了它们的工作原理和使用方式，还介绍了隐藏在这些功能背后的研究背景与设计模式。

然而，在开发、测试以及调试多线程的程序时仍然存在巨大的困难。开发人员很容易编写出一些看似能正常工作，但在一些情况下仍然会失败的程序（包括在正式发布的产品中，以及在高负载环境中）。本书不仅讲解了并发的理论基础，还介绍了各种实际的开发技术，这些知识对于构建可靠的、可伸缩的以及可维护的并发应用程序来说非常有用。本书并不仅是简单地罗列出各种并发API以及机制，而是详细地介绍了许多设计原则、设计模式以及思维模式，这些内容使得开发人员更容易构建出正确的并且高性能的并发程序。

本书主要内容包括：

- 并发性与线程安全性的基本概念
- 构建以及组合各种线程安全类的技术
- 使用java.util.concurrent包中的各种并发构建基础模块
- 性能优化中的注意事项
- 如何测试并发程序
- 以及一些高级主题，包括原子变量、无阻塞算法以及Java内存模型

上架指导：计算机/程序设计
ISBN 978-7-111-37004-8
9 787111 370048
定价：69.00元

客服热线：(010) 88378991, 88361066
购书热线：(010) 68326294, 88379649, 68995259
投稿热线：(010) 88379604
读者信箱：hzjsj@hzbook.com
华章网站 http://www.hzbook.com

网上购书：www.china-pub.com

PEARSON
www.pearson.com
封面设计 范华明

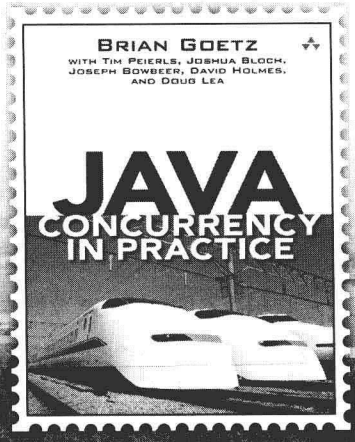
华章专业开发者丛书

Java 并发编程实战

Java Concurrency in Practice

（美） Brian Goetz
Tim Peierls
Joshua Bloch
Joseph Bowbeer
David Holmes
Doug Lea
著

童云兰 等译



机械工业出版社
China Machine Press

本书深入浅出地介绍了Java线程和并发，是一本完美的Java并发参考手册。书中从并发性和线程安全性的基本概念出发，介绍了如何使用类库提供的基本并发构建块，用于避免并发危险、构造线程安全的类及验证线程安全的规则，如何将小的线程安全类组合成更大的线程安全类，如何利用线程来提高并发应用程序的吞吐量，如何识别可并行执行的任务，如何提高单线程子系统的响应性，如何确保并发程序执行预期任务，如何提高并发代码的性能和可伸缩性等内容，最后介绍了一些高级主题，如显式锁、原子变量、非阻塞算法以及如何开发自定义的同步工具类。

本书适合Java程序开发人员阅读。

Authorized translation from the English language edition, entitled *Java Concurrency in Practice*, 9780321349606 by Brian Goetz, with Tim Peierls. et al., published by Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD., and CHINA MACHINE PRESS Copyright © 2012.

本书封底贴有Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

封底无防伪标均为盗版

版权所有，侵权必究

本书法律顾问 北京市展达律师事务所

本书版权登记号：图字：01-2011-1513

图书在版编目（CIP）数据

Java并发编程实战 /（美）盖茨（Goetz, B.）等著；童云兰等译．—北京：机械工业出版社，2012.2

（华章专业开发者丛书）

书名原文：Java Concurrency in Practice

ISBN 978-7-111-37004-8

I. J… II. ①盖… ②童… III. JAVA语言－程序设计 IV. TP312

中国版本图书馆CIP数据核字（2011）第281977号

机械工业出版社（北京市西城区百万庄大街22号 邮政编码 100037）

责任编辑：关 敏

北京市荣盛彩色印刷有限公司印刷

2012年2月第1版第1次印刷

186mm×240mm • 19.5印张

标准书号：ISBN 978-7-111-37004-8

定价：69.00元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换

客服热线：（010）88378991；88361066

购书热线：（010）68326294；88376949；68995259

投稿热线：（010）88379604

读者信箱：hzjsj@hzbook.com

对本书的赞誉

“我曾有幸在一个伟大的团队中工作，参与设计和实现在 Java 5.0 和 Java 6 等平台中新增的并发功能。现在，仍然是这个团队，将透彻地讲解这些新功能，以及关于并发的一般性概念。并发已不再只是高级用户谈论的话题，每一位 Java 开发人员都应该阅读这本书。”

——Martin Buchholz，Sun 公司的 JDK 并发大师

“在过去 30 多年时间里，计算机性能一直遵循着摩尔定律，但从现在开始，它将遵循 Amdahl 定律。编写能高效利用多处理器的代码非常具有挑战性。 在这本书中介绍的一些概念和技术，对于在当前（以及未来的）系统上编写安全的和可伸缩的代码来说都是非常有用的。”

——Doron Rajwan，Intel 公司研究人员

“如果你正在编写、设计、调试、维护以及分析多线程的 Java 程序，那么本书正是你所需要的。如果你曾对某个方法进行过同步，但却不理解其中的原因，那么你以及你的用户都有必要从头至尾仔细地读一读这本书。”

——Ted Neward，《Effective Enterprise Java》的作者

“Brian 非常清晰地阐述了并发的一些基本问题与复杂性。对于使用线程并关注程序执行性能的开发人员来说，这是一本必读的书。”

——Kirk Pepperdine，JavaPerformanceTuning.com 网站 CTO

“本书深入浅出地介绍了一些复杂的编程主题，是一本完美的 Java 并发参考手册。书中的每一页都包含了程序员日常需要应对的问题（以及相应的解决方案）。随着摩尔定律的发展趋势由提高处理器核的速度转向增加处理器核的数量，如何有效地利用并发性已变得越来越重要，本书正好介绍了这些方面的内容。”

——Cliff Click 博士，Azul Systems 公司高级软件工程师

“我对并发有着浓厚的兴趣，并且与大多数程序员相比，我或许写过更多存在线程死锁的代码，也在同步上犯了更多的错误。在介绍 Java 线程和并发等主题的众多书籍中，Brian 的这本书最具可读性，它通过循序渐进的方式将一些复杂的主题阐述得很清楚。我将本书推荐给 Java Specialists' Newsletter 的所有读者，因为它不仅有趣，而且很有用，它介绍了当前 Java 开发人员正在面对的许多问题。”

——Heinz Kabutz 博士，Java Specialists' Newsletter 的维护者

“我一直努力想使一些简单的问题变得更简单，然而本书已经简化了一个复杂但却关键的主

题：并发。这本书采用了创新的讲解方法、简单明了的风格，它注定会成为一本非常重要的书。

——Bruce Tate，《Beyond Java》的作者

这本书为 Java 开发人员在线程编程领域提供了不可多得的知识。我在读这本书时受到了极大的启发，部分原因在于它详细地介绍了 Java 中并发领域的 API，但更重要的却在于这本书以一种透彻并且易懂的方式来介绍复杂的并发知识，这是其他书籍很难媲美的。

——Bill Venners，《Inside the Java Virtual Machine》的作者

译 者 序

并发编程是Java语言的重要特性之一，在Java平台上提供了许多基本的并发功能来辅助开发多线程应用程序。然而，这些相对底层的并发功能与上层应用程序的并发语义之间并不存在一种简单而直观的映射关系。因此，如何在Java并发应用程序中正确且高效地使用这些功能就成了Java开发人员的关注重点。

本书正是为了解决这个问题而写的。书中采用循序渐进的讲解方式，从并发编程的基本理论入手，逐步介绍了在设计Java并发程序时各种重要的设计原则、设计模式以及思维模式，同时辅以丰富的示例代码作为对照和补充，使得开发人员能够更快地领悟Java并发编程的要领，围绕着Java平台的基础并发功能快速地构建大规模的并发应用程序。

全书内容由浅入深，共分为四个部分。第一部分介绍了Java并发编程的基础理论，包括线程安全性与状态对象的基础知识，如何构造线程安全的类并将多个小型的线程安全类构建成更大型的线程安全类，以及Java平台库中的一些基础并发模块；第二部分介绍了并发应用程序的构造理论，包括应用程序中并行语义的分解及其与逻辑任务的映射，任务的取消与关闭等行为的实现，以及Java线程池中的一些高级功能，此外还介绍了如何提高GUI应用程序的响应性；第三部分介绍了并发编程的性能调优，包括如何避免活跃性问题，如何提高并发代码的性能和可伸缩性以获得理想的性能，以及在测试并发代码正确性和性能时的一些实用技术；第四部分介绍了Java并发编程中的一些高级主题，包括显式锁、原子变量、非阻塞算法以及如何开发自定义的同步工具类等。

本书的特点在于注重阐述并发技术背后的理论知识，对于每种技术的介绍不仅使读者能做到“知其然”，更能做到“知其所以然”。对于希望深入研究和探索Java并发编程的读者来说，本书是非常合适的。

参与本书翻译工作的还有李杨、吴汉平、徐光景、童胜汉、陈军、胡凯、刘红、张玮、陈红、李斌、李勇涛、王海涛、周云波、彭敏才、张世锋、朱介秋、宗敬、李静、叶锦、高波、熊莉、程凤、陈娟、胡世娟、董敏、谢路阳、冯卓、李志勇、胡欢、王进等。由于译者的时间和水平有限，翻译中的疏漏和错误在所难免，还望读者和同行不吝指正。

童云兰

2011年11月于武汉

前　言

在写作本书时，对于中端桌面系统来说，多核处理器正变得越来越便宜。无独有偶，许多开发团队也注意到，在他们的项目中出现了越来越多与线程有关的错误报告。在 NetBeans 开发者网站上的最近一次公告中，一位核心维护人员注意到，为了修复与线程相关的问题，在某个类中竟然打了 14 次补丁。Dion Almaer，这位 TheServerSide 网站的前编辑，最近（在经过一番痛苦的调试过程并最终发现了一个与线程有关的错误之后）在其博客上写道，在大多数 Java 程序中充满了各种并发错误，使得程序只有在“偶然的情况下”才能正常工作。

确实，在开发、测试以及调试多线程程序时存在着巨大的困难，因为并发性错误通常并不会以某种确定的方式显现出来。当这些错误出现时，通常是在最糟糕的时刻，例如在正式产品中，或者在高负载的情况下。

当开发 Java 并发程序时，所要面对的挑战之一就是：平台提供的各种并发功能与开发人员在程序中需要的并发语义并不匹配。在 Java 语言中提供了一些底层机制，例如同步和条件等待，但在使用这些机制来实现应用级的协议与策略时必须始终保持一致。如果没有这些策略，那么在编写程序时，虽然程序看似能顺利地编译和运行，但却总会出现各种奇怪的问题。许多介绍并发的其他书籍更侧重于介绍一些底层机制和 API，而在设计级的策略和模式上叙述的不多。

Java 5.0 在 Java 并发应用程序的开发方面进展巨大，它不仅提供了一些新的高层组件，还补充了一些底层机制，从而使得无论是新手级开发人员还是专家级开发人员都能够更容易地构建并发应用程序。本书的作者都是 JCP 专家组的主要成员，也正是该专家组编写了这些新功能。本书不仅描述了这些新功能的行为和特性，还介绍了它们的底层设计模式和促使它们被添加到平台库中的应用场景。

我们的目标是向读者介绍一些设计规则和思维模式，从而使读者能够更容易也更乐意去构建正确的以及高性能的 Java 并发类和应用程序。

我们希望你能享受本书的阅读过程。

Brian Goetz
Williston，VT
2006 年 3 月

如何使用本书

为了解决在 Java 底层机制与设计级策略之间的不匹配问题，我们给出了一组简化的并发程序编写规则。专家看到这些规则会说：“嗯，这并不是完整的规则集。即使类 C 违背了规则 R，它仍然是线程安全的。”虽然在违背一些规则的情况下仍有可能编写出正确的并发程序，但这需

要对 Java 内存模型的底层细节有着深入的理解，而我们希望开发人员无须掌握这些细节就能编写出正确的并发程序。只要始终遵循这组简单的规则，就能编写出正确的并且可维护的并发程序。

我们假设读者对 Java 的基本并发机制已经有了一定程度的了解。本书并非是对并发的入门介绍——要了解这方面的内容，请参考其他书籍中有关线程的内容，例如《The Java Programming Language》（Arnold 等，2005）。此外，本书也不是介绍并发的百科全书——要了解这方面的内容，请参考《Concurrent Programming in Java》（Lea，2000）。事实上，本书提供了各种实用的设计规则，用于帮助开发人员创建安全的和高性能的并发类。在本书中相应的地方引用了以下书籍中的相关章节：《The Java Programming Language》、《Concurrent Programming in Java》、《The Java Language Specification》（Gosling 等，2005）以及《Effective Java》（Bloch, 2001），并分别使用 [JPL n.m]、[CPJ n.m]、[JLS n.m] 和 [EJ Item n] 来表示它们。

在进行简要的介绍（第 1 章）之后，本书共分为四个部分：

基础知识。第一部分（第 2 章～第 5 章）重点介绍了并发性和线程安全性的基本概念，以及如何使用类库提供的基本并发构建块来构建线程安全类。在第一部分给出了一个清单，其中总结了这一部分中介绍的最重要的规则。

第 2 章与第 3 章构成了本书的基础。在这两章中给出了几乎所有用于避免并发危险、构造线程安全的类以及验证线程安全的规则。如果读者重“实践”而轻“理论”，那么可能会直接跳到第二部分，但在开始编写任何并发代码之前，一定要回来读一读这两章！

第 4 章介绍了如何将一些小的线程安全类组合成更大的线程安全类。第 5 章介绍了在平台库中提供的一些基础的并发构建模块，包括线程安全的容器类和同步工具类。

结构化并发应用程序。第二部分（第 6 章～第 9 章）介绍了如何利用线程来提高并发应用程序的吞吐量或响应性。第 6 章介绍了如何识别可并行执行的任务，以及如何在任务执行框架中执行它们。第 7 章介绍了如何使任务和线程在执行完正常工作之前提前结束。在健壮的并发应用程序与看似能正常工作的应用程序之间存在的重要差异之一就是，如何实现取消以及关闭等操作。第 8 章介绍了任务执行框架中的一些更高级特性。第 9 章介绍了如何提高单线程子系统的响应性。

活跃性、性能与测试。第三部分（第 10 章～第 12 章）介绍了如何确保并发程序执行预期的任务，以及如何获得理想的性能。第 10 章介绍了如何避免一些使程序无法执行下去的活跃性故障。第 11 章介绍了如何提高并发代码的性能和可伸缩性。第 12 章介绍了在测试并发代码的正确性和性能时可以采用的一些技术。

高级主题。第四部分（第 13 章～第 16 章）介绍了资深开发人员可能感兴趣的一些主题，包括：显式锁、原子变量、非阻塞算法以及如何开发自定义的同步工具类。

代码示例

虽然书中很多一般性的概念同样适用于 Java 5.0 之前的版本以及一些非 Java 的运行环境，但其中大多数示例代码（以及关于 Java 内存模型的所有描述）是基于 Java 5.0 或更高版本的，而且某些代码示例中还使用了 Java 6 的一些新增功能。

我们对书中的代码示例已经进行了压缩，以便减少代码量并重点突出与内容相关的部分。在本书的网站 http://www.javaconcurrencyinpractice.com 上提供了完整的代码示例、辅助示例以及勘误表。

代码示例可分为三类："好的"示例、"一般的"示例和"糟糕的"示例。"好的"示例是应该被效仿的技术。"糟糕的"示例是一定不能效仿的技术，而且还会用一个"Mr. Yuk"的图标[⊖]来表示该示例中的代码是"有害的"（参见程序清单 1）。"一般的"示例给出的技术并不一定是错的，但却是脆弱的、有风险的或是性能较差的，并且会用一个"Mr.Could Be Happier"图标来表示，如程序清单 2 所示。

程序清单 1　糟糕的链表排序方式（不要这样做）

```
public <T extends Comparable<? super T>> void sort(List<T> list) {
// 永远不要返回错误的答案！
    System.exit(0);
}
```

有些读者会质疑这些"糟糕的"示例在本书中的作用，毕竟，在一本书中应该给出如何做正确的事，而不是错误的事。这些"糟糕的"示例有两个目的，它们揭示了一些常见的缺陷，但更重要的是它们示范了如何分析程序的线程安全性，而要实现这个目的，最佳的方式就是观察线程安全性是如何被破坏的。

程序清单 2　非最优方式的链表排序

```
public <T extends Comparable<? super T>> void sort(List<T> list) {
    for (int i=0; i<1000000; i++)
        doNothing();
    Collections.sort(list);
}
```

致谢

本书诞生于 Java Community Process JSR 166 为 Java 5.0 开发 java.util.concurrent 包的过程中。还有许多人参与到 JSR 166 中，特别感谢 Martin Buchholz 将全部的工作融入到 JDK 中，并感谢 concurrency-interest 邮件列表中的所有读者对 API 草案提出的建议和反馈。

有不少来自各方面的人员都提出了建议和帮助，使得本书的内容得到了极大的充实。感谢 Dion Almaer、Tracy Bialik、Cindy Bloch、Martin Buchholz、Paul Christmann、Cliff Click、Stuart Halloway、David Hovemeyer、Jason Hunter、Michael Hunter、Jeremy Hylton、Heinz Kabutz、Robert Kuhar、Ramnivas Laddad、Jared Levy、Nicole Lewis、Victor Luchangco、Jeremy Manson、Paul Martin、Berna Massingill、Michael Maurer、Ted Neward、Kirk Pepperdine、Bill Pugh、Sam Pullara、Russ Rufer、Bill Scherer、Jeffrey Siegal、Bruce Tate、Gil Tene、Paul Tyma，以及硅谷模

⊖ Mr. Yuk 是匹兹堡儿童医院的注册商标，本书获得了该商标的授权，因此可以在本书中使用。

式小组的所有成员，他们通过各种技术交流为本书提供了指导建议，使得本书更加完善。

特别感谢 Cliff Biffle、Barry Hayes、Dawid Kurzyniec、Angelika Langer、Doron Rajwan 和 Bill Venners，他们非常仔细地审阅了本书的全稿，指出了代码示例中的错误，并提出了大量的改进建议。

感谢 Katrina Avery 的编辑工作，以及 Rosemary Simpson 在非常短的时间里完成了索引生成工作。感谢 Ami Dewar 绘制的插图。

感谢 Addison-Wesley 的全体成员，他们使本书得以最终问世。Ann Sellers 启动了编写本书的项目，Greg Doench 监督并帮助本书有条不紊地完成，Elizabeth Ryan 负责本书的出版过程。

此外还要感谢许许多多的软件工程师，他们开发了本书得以依赖的各种软件，这些软件包括 TEX、LATEX、Adobe Acrobat、pic、grap、Adobe Illustrator、Perl、Apache Ant、IntelliJ IDEA、GNU emacs、Subversion、TortoiseSVN，当然，还有 Java 平台及其类库。

目　录

对本书的赞誉
译者序
前　言

第 1 章　简介 ……1
1.1　并发简史……1
1.2　线程的优势……2
1.2.1　发挥多处理器的强大能力 ……2
1.2.2　建模的简单性……3
1.2.3　异步事件的简化处理……3
1.2.4　响应更灵敏的用户界面……4
1.3　线程带来的风险……4
1.3.1　安全性问题 ……5
1.3.2　活跃性问题 ……7
1.3.3　性能问题……7
1.4　线程无处不在 ……7

第一部分　基础知识

第 2 章　线程安全性 ……11
2.1　什么是线程安全性……13
2.2　原子性 ……14
2.2.1　竞态条件 ……15
2.2.2　示例：延迟初始化中的竞态条件……16
2.2.3　复合操作……17
2.3　加锁机制……18
2.3.1　内置锁……20
2.3.2　重入 ……21
2.4　用锁来保护状态……22
2.5　活跃性与性能 ……23
第 3 章　对象的共享 ……27
3.1　可见性 ……27
3.1.1　失效数据……28
3.1.2　非原子的 64 位操作 ……29
3.1.3　加锁与可见性……30
3.1.4　Volatile 变量……30
3.2　发布与逸出……32
3.3　线程封闭……35
3.3.1　Ad-hoc 线程封闭……35
3.3.2　栈封闭……36
3.3.3　ThreadLocal 类……37
3.4　不变性 ……38
3.4.1　Final 域 ……39
3.4.2　示例：使用 Volatile 类型来发布不可变对象……40
3.5　安全发布……41
3.5.1　不正确的发布：正确的对象被破坏……42
3.5.2　不可变对象与初始化安全性……42
3.5.3　安全发布的常用模式……43
3.5.4　事实不可变对象 ……44
3.5.5　可变对象 ……44
3.5.6　安全地共享对象 ……44
第 4 章　对象的组合 ……46
4.1　设计线程安全的类……46
4.1.1　收集同步需求……47
4.1.2　依赖状态的操作……48

4.1.3 状态的所有权……48
4.2 实例封闭……49
4.2.1 Java 监视器模式……51
4.2.2 示例：车辆追踪……51
4.3 线程安全性的委托……53
4.3.1 示例：基于委托的车辆追踪器……54
4.3.2 独立的状态变量……55
4.3.3 当委托失效时……56
4.3.4 发布底层的状态变量……57
4.3.5 示例：发布状态的车辆追踪器……58
4.4 在现有的线程安全类中添加功能……59
4.4.1 客户端加锁机制……60
4.4.2 组合……62
4.5 将同步策略文档化……62
第 5 章 基础构建模块……66
5.1 同步容器类……66
5.1.1 同步容器类的问题……66
5.1.2 迭代器与 Concurrent-ModificationException……68
5.1.3 隐藏迭代器……69
5.2 并发容器……70
5.2.1 ConcurrentHashMap……71
5.2.2 额外的原子 Map 操作……72
5.2.3 CopyOnWriteArrayList……72
5.3 阻塞队列和生产者－消费者模式……73
5.3.1 示例：桌面搜索……75
5.3.2 串行线程封闭……76
5.3.3 双端队列与工作密取……77
5.4 阻塞方法与中断方法……77
5.5 同步工具类……78
5.5.1 闭锁……79
5.5.2 FutureTask……80
5.5.3 信号量……82
5.5.4 栅栏……83
5.6 构建高效且可伸缩的结果缓存……85

第二部分 结构化并发应用程序

第 6 章 任务执行……93
6.1 在线程中执行任务……93
6.1.1 串行地执行任务……94
6.1.2 显式地为任务创建线程……94
6.1.3 无限制创建线程的不足……95
6.2 Executor 框架……96
6.2.1 示例：基于 Executor 的 Web 服务器……97
6.2.2 执行策略……98
6.2.3 线程池……98
6.2.4 Executor 的生命周期……99
6.2.5 延迟任务与周期任务……101
6.3 找出可利用的并行性……102
6.3.1 示例：串行的页面渲染器……102
6.3.2 携带结果的任务 Callable 与 Future……103
6.3.3 示例：使用 Future 实现页面渲染器……104
6.3.4 在异构任务并行化中存在的局限……106
6.3.5 CompletionService:Executor 与 BlockingQueue……106
6.3.6 示例：使用 CompletionService 实现页面渲染器……107
6.3.7 为任务设置时限……108
6.3.8 示例：旅行预定门户网站……109
第 7 章 取消与关闭……111
7.1 任务取消……111
7.1.1 中断……113

7.1.2 中断策略……116
7.1.3 响应中断……117
7.1.4 示例：计时运行……118
7.1.5 通过 Future 来实现取消……120
7.1.6 处理不可中断的阻塞……121
7.1.7 采用 newTaskFor 来封装非标准的取消……122
7.2 停止基于线程的服务……124
7.2.1 示例：日志服务……124
7.2.2 关闭 ExecutorService……127
7.2.3 “毒丸”对象……128
7.2.4 示例：只执行一次的服务……129
7.2.5 shutdownNow 的局限性……130
7.3 处理非正常的线程终止……132
7.4 JVM 关闭……135
7.4.1 关闭钩子……135
7.4.2 守护线程……136
7.4.3 终结器……136
第 8 章 线程池的使用……138
8.1 在任务与执行策略之间的隐性耦合……138
8.1.1 线程饥饿死锁……139
8.1.2 运行时间较长的任务……140
8.2 设置线程池的大小……140
8.3 配置 ThreadPoolExecutor……141
8.3.1 线程的创建与销毁……142
8.3.2 管理队列任务……142
8.3.3 饱和策略……144
8.3.4 线程工厂……146
8.3.5 在调用构造函数后再定制 ThreadPoolExecutor……147
8.4 扩展 ThreadPoolExecutor……148
8.5 递归算法的并行化……149
第 9 章 图形用户界面应用程序……156
9.1 为什么 GUI 是单线程的……156
9.1.1 串行事件处理……157
9.1.2 Swing 中的线程封闭机制……158
9.2 短时间的 GUI 任务……160
9.3 长时间的 GUI 任务……161
9.3.1 取消……162
9.3.2 进度标识和完成标识……163
9.3.3 SwingWorker……165
9.4 共享数据模型……165
9.4.1 线程安全的数据模型……166
9.4.2 分解数据模型……166
9.5 其他形式的单线程子系统……167

第三部分 活跃性、性能与测试

第 10 章 避免活跃性危险……169
10.1 死锁……169
10.1.1 锁顺序死锁……170
10.1.2 动态的锁顺序死锁……171
10.1.3 在协作对象之间发生的死锁……174
10.1.4 开放调用……175
10.1.5 资源死锁……177
10.2 死锁的避免与诊断……178
10.2.1 支持定时的锁……178
10.2.2 通过线程转储信息来分析死锁……178
10.3 其他活跃性危险……180
10.3.1 饥饿……180
10.3.2 糟糕的响应性……181
10.3.3 活锁……181
第 11 章 性能与可伸缩性……183
11.1 对性能的思考……183
11.1.1 性能与可伸缩性……184
11.1.2 评估各种性能权衡因素……185
11.2 Amdahl 定律……186

11.2.1 示例：在各种框架中隐藏的串行部分……188
11.2.2 Amdahl 定律的应用……189
11.3 线程引入的开销……189
11.3.1 上下文切换……190
11.3.2 内存同步……190
11.3.3 阻塞……192
11.4 减少锁的竞争……192
11.4.1 缩小锁的范围（“快进快出”）……193
11.4.2 减小锁的粒度……195
11.4.3 锁分段……196
11.4.4 避免热点域……197
11.4.5 一些替代独占锁的方法……198
11.4.6 监测 CPU 的利用率……199
11.4.7 向对象池说“不”……200
11.5 示例：比较 Map 的性能……200
11.6 减少上下文切换的开销……201
第 12 章 并发程序的测试……204
12.1 正确性测试……205
12.1.1 基本的单元测试……206
12.1.2 对阻塞操作的测试……207
12.1.3 安全性测试……208
12.1.4 资源管理的测试……212
12.1.5 使用回调……213
12.1.6 产生更多的交替操作……214
12.2 性能测试……215
12.2.1 在 PutTakeTest 中增加计时功能……215
12.2.2 多种算法的比较……217
12.2.3 响应性衡量……218
12.3 避免性能测试的陷阱……220
12.3.1 垃圾回收……220
12.3.2 动态编译……220
12.3.3 对代码路径的不真实采样……222
12.3.4 不真实的竞争程度……222
12.3.5 无用代码的消除……223
12.4 其他的测试方法……224
12.4.1 代码审查……224
12.4.2 静态分析工具……224
12.4.3 面向方面的测试技术……226
12.4.4 分析与监测工具……226

第四部分 高级主题

第 13 章 显式锁……227
13.1 Lock 与 ReentrantLock……227
13.1.1 轮询锁与定时锁……228
13.1.2 可中断的锁获取操作……230
13.1.3 非块结构的加锁……231
13.2 性能考虑因素……231
13.3 公平性……232
13.4 在 synchronized 和 ReentrantLock 之间进行选择……234
13.5 读－写锁……235
第 14 章 构建自定义的同步工具……238
14.1 状态依赖性的管理……238
14.1.1 示例：将前提条件的失败传递给调用者……240
14.1.2 示例：通过轮询与休眠来实现简单的阻塞……241
14.1.3 条件队列……243
14.2 使用条件队列……244
14.2.1 条件谓词……244
14.2.2 过早唤醒……245
14.2.3 丢失的信号……246
14.2.4 通知……247
14.2.5 示例：阀门类……248
14.2.6 子类的安全问题……249

14.2.7 封装条件队列……250
14.2.8 入口协议与出口协议……250
14.3 显式的 Condition 对象……251
14.4 Synchronizer 剖析……253
14.5 AbstractQueuedSynchronizer……254
14.6 java.util.concurrent 同步器类中的 AQS……257
14.6.1 ReentrantLock……257
14.6.2 Semaphore 与 CountDownLatch……258
14.6.3 FutureTask……259
14.6.4 ReentrantReadWriteLock……259
第 15 章 原子变量与非阻塞同步机制……261
15.1 锁的劣势……261
15.2 硬件对并发的支持……262
15.2.1 比较并交换……263
15.2.2 非阻塞的计数器……264
15.2.3 JVM 对 CAS 的支持……265
15.3 原子变量类……265
15.3.1 原子变量是一种“更好的 volatile”……266
15.3.2 性能比较：锁与原子变量……267
15.4 非阻塞算法……270
15.4.1 非阻塞的栈……270
15.4.2 非阻塞的链表……272
15.4.3 原子的域更新器……274
15.4.4 ABA 问题……275
第 16 章 Java 内存模型……277
16.1 什么是内存模型，为什么需要它……277
16.1.1 平台的内存模型……278
16.1.2 重排序……278
16.1.3 Java 内存模型简介……280
16.1.4 借助同步……281
16.2 发布……283
16.2.1 不安全的发布……283
16.2.2 安全的发布……284
16.2.3 安全初始化模式……284
16.2.4 双重检查加锁……286
16.3 初始化过程中的安全性……287
附录 A 并发性标注……289
参考文献……291